

Doc. no. P1001R2
Date: 2019-02-22
Project: Programming Language C++
Audience: SG1 Parallelism and Concurrency
Library Working Group
Reply to: Alisdair Meredith <ameredith1@bloomberg.net>
Pablo Halpern <phalpern@halpernwrightsoftware.com>

Target Vectorization Policies from Parallelism V2 TS to C++20

Table of Contents

- 0. [Revision History](#)
 - [Revision 0](#)
 - [Revision 1](#)
 - [Revision 2](#)
- 1. [Introduction](#)
- 2. [Stating the problem](#)
 - 1. [Conclusion of SG1 Review \(Jacksonville 2018\)](#)
- 3. [Propose Solution](#)
- 4. [Other Directions](#)
- 5. [Formal Wording](#)
- 6. [Acknowledgements](#)
- 7. [References](#)

Revision History

Revision 0

Original version of the paper for the 2018 Jacksonville meeting.

Revision 1

Revision of the paper following SG1 feedback at the 2018 Jacksonville meeting, proposing just the unsquenced policy for C++20.

Revision 2

Revision of the paper following LWG feedback at the 2018 San Diego meeting.

- Rebased the wording onto [N4800](#).
- Proposed feature macros for the <version> header.
- Moved normative permissions on using the new policy to [algorithms.parallel.exec].
- Moved the IS definition of a vector-unsafe function up two subclauses into the parallel algorithms terms and definitions subclause.

1 Introduction

The Parallelism v2 PDTS has two new vector policies for the parallel algorithms that might better target C++20 directly.

2 Stating the problem

We sent the Parallelism V2 working paper to PDTS ballot at the Jacksonville meeting. It seems reasonable to conclude that if we want feedback from the TS process, then anything in that TS vehicle would miss the merge deadline for C++20. This paper was written to confirm early that there were no features more appropriately targeted straight at the IS.

Of the (expected) three components to that TS, the Task Blocks feature depends on the `exception_list` class that is still underspecified for general use, and waiting feedback from such a TS process.

The `simd` library type has been going through rapid evolution and been subject to much change in the review process, so seems appropriate to seek feedback through the TS process.

Finally, there are two new vectorization policies for the parallel algorithms, which rely on the main `<algorithm>` header providing overloads for the new policies in the non-experimental `std` namespace in order to be usable. The algorithms can safely be implemented as reverting to serial or any other lesser parallel behavior without using the freedoms granted by the wavefront policies, so implementability is much less of a concern than QoI. There is no room in the design space for how we dispatch to the algorithms to be seeking TS feedback, so it seems reasonable to suggest that if we want this feature, it would be more appropriate to target the C++20 standard directly, avoiding the awkward interaction with the `std` and `experimental` namespaces. We are still early enough in the C++20 cycle to respond to unexpected implementer feedback, and the current state of the C++17 parallel algorithms is that most standard library vendors are still starting or in the middle of their first implementation of the parallel algorithms, so it would be easier to tackle these extra policies now along with the rest of that work.

2.1 Conclusion of SG1 Review (Jacksonville 2018)

Merging the unsequenced policy was entirely non-controversial, and is recommended for C++20 by SG1. It fills an important missing hole in the original specification.

Moving ahead with the vector wavefront policy seemed premature to the group. There is a desire to see that the new policy is generally useful, and provides a real optimization opportunity in practice. There are particular concerns that it appears useful in only a small subset of the parallel algorithms, and a more targeted approach for those few special cases might be more appropriate.

3 Propose Solution

Merge the wording for the unsequenced execution policy into C++20, but leave it in place in the Parallelism V2 TS as it may be useful as a fallback policy for algorithms implementing the vector execution policy.

4 Other Directions

The original proposal recommended merging both vectorization policies, and their associated machinery, into C++20, and removing them from the Parallelism V2 TS. SG1 rejected this approach as there is a desire to see real benefits from the implementation of the vector wavefront policy in shipping libraries before adopting in the standard. There was a particular concern that very few existing algorithms are expected to see a benefit from this policy, other than the new algorithms specifically added in the TS for that policy. We may want to revisit the idea of whether that policy is a generic execution policy like the others, or whether we add just a few specific overloads explicitly taking this policy for the few algorithms that are expected to profit.

5 Formal Wording

16.3.1 General [support.limits.general]

Table 36 — Standard library feature-test macros

Macro name	Value	Header(s)
------------	-------	-----------

```
...
__cpp_lib_execution 201603L 201903L <execution>
...
```

The value of the macro is a suggestion, the project editor should pick a final value inspired by the date the edits are applied.

19.18.2 Header <execution> synopsis [execution.syn]

```
namespace std {
    // 19.18.3, execution policy type trait
    template<class T> struct is_execution_policy;
    template<class T> inline constexpr bool is_execution_policy_v = is_execution_policy<T>::value;
}

namespace std::execution {
    // 19.18.4, sequenced execution policy
    class sequenced_policy;

    // 19.18.5, parallel execution policy
    class parallel_policy;

    // 19.18.6, parallel and unsequenced execution policy
    class parallel_unsequenced_policy;

    // 19.18.7, unsequenced execution policy
    class unsequenced_policy;

    // 19.18.78, execution policy objects
    inline constexpr sequenced_policy seq{ unspecified };
    inline constexpr parallel_policy par{ unspecified };
    inline constexpr parallel_unsequenced_policy par_unseq{ unspecified };
    inline constexpr unsequenced_policy unseq{ unspecified };
}
```

19.18.7 Unsequenced execution policy [parallel.execpol.unseq]

```
class unsequenced_policy{ unspecified };
```

1. The class unsequenced_policy is an execution policy type used as a unique type to disambiguate parallel algorithm overloading and indicate that a parallel algorithm's execution may be vectorized, e.g., executed on a single thread using instructions that operate on multiple data items.
2. During the execution of a parallel algorithm with the execution::unsequenced_policy policy, if the invocation of an element access function exits via an uncaught exception, terminate(.) shall be called.

19.18.78 Execution policy objects [execpol.objects]

```
inline constexpr execution::sequenced_policy      execution::seq{ unspecified };
inline constexpr execution::parallel_policy        execution::par{ unspecified };
inline constexpr execution::parallel_unsequenced_policy execution::par_unseq{ unspecified };
inline constexpr execution::unsequenced_policy     execution::unseq{ unspecified };
```

1. The header <execution> declares global objects associated with each type of execution policy.

24.3.1 Terms and definitions [algorithms.parallel.defns]

3. A standard library function is *vectorization-unsafe* if it is specified to synchronize with another function invocation, or another function invocation is specified to synchronize with it, and if it is not a memory allocation or deallocation function. [Note: Implementations must ensure that internal synchronization inside standard library functions does not prevent forward progress when those functions are executed by threads of execution with weakly parallel forward progress guarantees. —

endnote]. Example:

```
int x = 0;
std::mutex m;
int a[] = {1,2};
std::for_each(std::execution::par_unseq, std::begin(a), std::end(a), [&](int) {
    std::lock_guard<mutex> guard(m); // incorrect: lock_guard constructor calls m.lock()
    ++x;
});
```

The above program may result in two consecutive calls to `m.lock()` on the same thread of execution (which may deadlock), because the applications of the function object are not guaranteed to run on different threads of execution. — end example]

24.3.3 Effect of execution policies on algorithm execution [algorithms.parallel.exec]

4. The invocations of element access functions in parallel algorithms invoked with an execution policy object of type `execution::sequenced_policy` all occur in the calling thread of execution. [Note: The invocations are not interleaved; see 6.8.1. — end note]
5. The invocations of element access functions in parallel algorithms invoked with an execution policy object of type `execution::unsequenced_policy` are permitted to execute in an unordered fashion in the calling thread of execution, unsequenced with respect to one another in the calling thread of execution. [Note: This means that multiple function object invocations may be interleaved on a single thread of execution, which overrides the usual guarantee from 6.8.1 [intro.execution] that function executions do not overlap with one another. — end note] Since `execution::unsequenced_policy` allows the execution of element access functions to be interleaved on a single thread of execution, blocking synchronization, including the use of mutexes, risks deadlock. Thus, the synchronization with `execution::unsequenced_policy` is restricted as follows: vectorization-unsafe standard library functions may not be invoked by user code called from `execution::unsequenced_policy` algorithms.
6. The invocations of element access functions in parallel algorithms invoked with an execution policy object of type `execution::parallel_policy` are permitted to ...
7. The invocations of element access functions in parallel algorithms invoked with an execution policy object of type `execution::parallel_unsequenced_policy` are permitted to execute in an unordered fashion in unspecified threads of execution, and unsequenced with respect to one another within each thread of execution. These threads of execution are either the invoking thread of execution or threads of execution implicitly created by the library; the latter will provide weakly parallel forward progress guarantees. [Note: This means that multiple function object invocations may be interleaved on a single thread of execution, which overrides the usual guarantee from 6.8.1 that function executions do not interleave/overlap with one another. — end note] Since `execution::parallel_unsequenced_policy` allows the execution of element access functions to be interleaved on a single thread of execution, blocking synchronization, including the use of mutexes, risks deadlock. Thus, the synchronization with `execution::parallel_unsequenced_policy` is restricted as follows: A standard library function is **vectorization-unsafe** if it is specified to synchronize with another function invocation, or another function invocation is specified to synchronize with it, and if it is not a memory allocation or deallocation function. Vectorization-unsafe standard library functions may not be invoked by user code called from `execution::parallel_unsequenced_policy` algorithms. [Note: Implementations must ensure that internal synchronization inside standard library functions does not prevent forward progress when those functions are executed by threads of execution with weakly parallel forward progress guarantees. — endnote] Example:

```
int x = 0;
std::mutex m;
int a[] = {1,2};
std::for_each(std::execution::par_unseq, std::begin(a), std::end(a), [&](int) {
    std::lock_guard<mutex> guard(m); // incorrect: lock_guard constructor calls m.lock()
    ++x;
});
```

The above program may result in two consecutive calls to `m.lock()` on the same thread of execution (which may deadlock), because the applications of the function object are not guaranteed to run on different threads of execution. — end example] [Note: The semantics of the

~~execution::parallel_policy or the execution::parallel_unsequenced_policy invocation allow the implementation to fall back to sequential execution if the system cannot parallelize an algorithm invocation due to lack of resources. — end note]~~

8. [Note: The semantics of invocation with execution::unsequenced_policy, execution::parallel_policy, or execution::parallel_unsequenced_policy allow the implementation to fall back to sequential execution if the system cannot parallelize an algorithm invocation, e.g., due to lack of resources. — end note]
9. If an invocation of a parallel algorithm uses threads of execution implicitly created by the library, then the invoking thread of execution will either ...

6 Acknowledgements

Thanks to all the troublemakers in Jacksonville who persuaded me there was time to write the original late paper! Thanks to SG1 for finding the time to review it, and provide feedback for a more appropriate timetable.

7 References

- [N4725](#) Working Draft, Technical Specification for C++ Extensions for Parallelism Version 2, Jared Hoberock
- [P0075R2](#) Template Library for Parallel For Loops, Pablo Halpern, Clark Nelson, Arch D. Robison, Robert Geva
- [P0076R4](#) Vector and Wavefront Policies, Arch Robison, Pablo Halpern, Robert Geva, Clark Nelson, Jens Maurer